

Laboratorio 3

Repositorio

[Link Repositorio](#)

Integrantes

Nombre	Carnet	Usuario Git
Edwin Jose Gabriel De Leon Garcia	22809	EJGDLG
Gustavo Adolfo Cruz Bardales	22779	G2309
Josué Emanuel Say Garcia	22801	JosueSay
Mathew Alexander Cordero Aquino	22982	donmatthiuz

Una empresa de desarrollo de videojuegos está evaluando el uso de agentes de RL para generar comportamiento no determinista en personajes de un juego de rol táctico. El entorno de prueba es un mapa de cuadrícula de 6×6 con zonas de recompensa, zonas de penalización y un estado terminal. El equipo técnico necesita entender si Monte Carlo es viable para este dominio antes de escalar a métodos más complejos, y quiere comparar su comportamiento contra la línea base de Programación Dinámica de la semana anterior.

Su grupo ha sido contratado para diseñar el experimento, implementar los métodos, analizar los resultados, y producir un reporte técnico con recomendaciones sobre la viabilidad de Monte Carlo para este dominio.

Task 1 (Entrega Parcial)

Antes de implementar nada, diseñen formalmente el experimento que van a ejecutar.

Inciso 1

Especificación del MDP: definan el espacio de estados, el espacio de acciones, la función de recompensa y el factor de descuento γ para el mapa 6×6 . Justifiquen cada decisión de diseño. El mapa debe incluir al menos dos zonas de recompensa positiva, una zona de penalización, y un estado terminal. La función de recompensa debe capturar al menos dos objetivos potencialmente conflictivos propios del dominio de videojuegos.

Respuesta:

El mapa es una cuadrícula de 6 filas por 6 columnas. Se usan coordenadas (f, c) con $f, c \in \{0, \dots, 5\}$, donde f crece hacia abajo y c hacia la derecha.

	c=0	c=1	c=2	c=3	c=4	c=5
f=0	S	.	.	.	.	.
f=1	.	.	.	.	T1	.
f=2	.	T2	.	.	.	.
f=3	.	.	.	P	.	.
f=4	.	.	.	.	.	.
f=5	.	.	.	.	.	G

Símbolo	Celda	Significado
S	(0, 0)	Punto de aparición del personaje
T1	(1, 4)	Zona de recompensa (cofre grande)
T2	(2, 1)	Zona de recompensa (cofre pequeño)
P	(3, 3)	Zona de penalización (trampa o zona patrullada)
G	(5, 5)	Estado terminal (salida de la misión)

Espacio de estados

$$s = (f, c, b_1, b_2), \quad f, c \in \{0, \dots, 5\}, \quad b_1, b_2 \in \{0, 1\}$$

donde b_1 y b_2 indican si el cofre T1 y el cofre T2 ya fueron recogidos en el episodio actual. Cardinalidad:

$$|S| = 36 \cdot 2 \cdot 2 = 144$$

Los dos bits no son decoración. Sin ellos el personaje puede salir de la celda del cofre y volver a entrar para cobrar la recompensa otra vez, y como el cofre da más de lo que cuestan los dos pasos del ciclo, la política óptima consistiría en dar vueltas sobre el cofre para siempre y nunca llegar a la salida. El episodio dejaría de terminar y la tarea dejaría de ser episódica, que es justo lo que Monte Carlo necesita. Con los bits, el cofre se cobra una sola vez por episodio, tal como funciona un cofre en un juego real. El estado sigue siendo Markov porque el bit resume todo lo que del pasado importa para decidir.

El estado inicial es $s_0 = (0, 0, 0, 0)$ y el estado terminal es cualquier estado con $(f, c) = (5, 5)$, que es absorbente. El episodio también se corta a los 200 pasos para que una política aleatoria no genere trayectorias infinitas.

Espacio de acciones

$$A = \{\text{arriba, abajo, izquierda, derecha}\}, \quad |A| = 4$$

Discreto porque el personaje se mueve por celdas, que es como se mueve una unidad en un juego de rol táctico. Las cuatro acciones están disponibles en todo estado. Si el movimiento sale del mapa, el personaje se queda donde está y el paso se consume igual.

Esto evita tener que definir $A(s)$ por estado y además le enseña al agente que chocar con el borde desperdicia un turno.

Las transiciones son deterministas: la acción elegida siempre mueve al personaje a la celda correspondiente. El comportamiento no determinista que pide el estudio no viene del entorno sino de la política ϵ -soft del Inciso 3, que es la que hace que dos partidas no se vean iguales.

Función de recompensa

Evento	Valor
Cada paso	-1
Entrar a T1 con $b_1 = 0$	+5
Entrar a T2 con $b_2 = 0$	+3
Entrar a P	-10
Llegar a G	+20

Las recompensas se suman cuando coinciden en el mismo paso. Por ejemplo, entrar a T1 por primera vez da $-1 + 5 = +4$.

Objetivos en conflicto

El diseño pone en tensión dos cosas que cualquier jugador reconoce: terminar la misión rápido y llevarse el botín. El costo de -1 por paso empuja hacia la salida y las zonas de recompensa empujan hacia el desvío. La trampa agrega un tercer eje, porque está sentada sobre la diagonal natural del mapa y la ruta corta tiene que rodearla.

Que el conflicto sea real se ve comparando el retorno de las cuatro rutas razonables, todas esquivando la trampa, con $\gamma = 0.95$:

Ruta	Pasos	Retorno G_0
Directa a G	10	4.58
Recoge T1 y sigue a G	10	8.65
Recoge T2 y sigue a G	10	7.29
Recoge T2, luego T1, y termina en G	12	8.57

Cada cofre por separado queda sobre una ruta que avanza siempre hacia la salida, así que recoger uno de los dos no cuesta ningún paso extra y siempre conviene. Recoger los dos sí cuesta: obliga a un desvío de 2 pasos y retrasa el +20. Con $\gamma = 0.95$ la mejor ruta es quedarse solo con T1 (8.65) y la segunda mejor es recoger ambos (8.57), una diferencia de 0.09. Es un empate casi perfecto, y es a propósito: sirve para ver si Monte Carlo, con su ruido de muestreo, logra distinguir dos rutas tan cercanas cuando Value Iteration las separa sin problema.

Factor de descuento

Se propone $\gamma = 0.95$. El horizonte efectivo es $1/(1 - \gamma) = 20$ pasos, y la ruta útil más larga del mapa es de 12 pasos, así que desde cualquier celda el agente todavía alcanza a ver el valor de la salida y no se vuelve miope. Además γ es la perilla que decide el estilo del personaje:

γ	Mejor ruta	Personaje que resulta
0.90	Solo T1 (4.52 contra 4.19)	Va a lo suyo y desprecia el desvío
0.95	Solo T1 (8.65 contra 8.57)	Casi indiferente entre correr y saquear
0.99	Ambos cofres (14.19 contra 13.51)	Saqueador, el desvío deja de importarle

La tarea es episódica y termina, así que $\gamma < 1$ no hace falta para que el retorno sea finito. Aquí γ se usa como parámetro de diseño de comportamiento, no como truco de convergencia, y se elige 0.95 porque es el punto donde las dos conductas quedan casi empatadas, que es el escenario más exigente para comparar los dos métodos.

Inciso 2

Selección de variante Monte Carlo: argumenten cuál variante usarán, First-Visit o Every-Visit, y por qué es más apropiada para este dominio específico. Incluyan en su argumento una discusión sobre la frecuencia esperada de visitas a estados en un mapa 6×6 bajo una política aleatoria.

Respuesta:

Se usará First-Visit Monte Carlo.

Frecuencia esperada de visitas

Bajo política uniforme aleatoria el personaje ejecuta una caminata aleatoria sobre una cuadrícula de 36 celdas con bordes reflejantes. Una caminata aleatoria en 2D es recurrente y no tiene ninguna preferencia por la salida, así que el tiempo esperado para llegar de $(0, 0)$ a $(5, 5)$ es del orden de cientos de pasos, no de decenas. Con episodios de unos 200 a 300 pasos repartidos entre 36 celdas, cada celda se visita en promedio del orden de 6 a 8 veces por episodio, y las celdas cercanas al punto de aparición bastante más porque la caminata tarda en alejarse. Las visitas no son un caso raro en este mapa, son la norma.

Por qué eso favorece a First-Visit

Every-Visit aprovecharía esas 6 u 8 visitas como 6 u 8 muestras, y a primera vista parece que multiplica los datos. El problema es que los retornos de varias visitas al mismo estado dentro del mismo episodio comparten la misma cola de recompensas: todos terminan con el mismo +20, todos pasan por los mismos cofres, todos cargan la misma trampa si el personaje cayó en ella. Son muestras fuertemente correlacionadas, así que n visitas valen mucho menos que n muestras independientes y la varianza baja mucho menos de lo que sugiere el conteo. Peor aún, esa correlación hace que la barra de error

calculada con la desviación muestral quede optimista, y como el objetivo del laboratorio es comparar contra Value Iteration, una barra de error mentirosa arruina la comparación.

First-Visit toma un solo retorno por estado y por episodio. Como los episodios son independientes entre sí, esos retornos sí son independientes e idénticamente distribuidos, el promedio es un estimador insesgado de $V^\pi(s)$ y su varianza cae limpiamente como σ^2/n con n igual al número de episodios en los que apareció el estado. Eso permite reportar intervalos de confianza válidos y decir con honestidad cuántos episodios hacen falta para igualar la precisión de Value Iteration.

El costo es que se descartan datos. Se acepta porque en un mapa de 36 celdas con transiciones deterministas simular un episodio es baratísimo: es más fácil correr más episodios que exprimir cada uno. Every-Visit sería la opción razonable si simular fuera caro, por ejemplo si cada episodio requiriera correr el motor del juego completo.

Inciso 3

Estrategia de exploración: argumenten si usarán Exploring Starts o política ϵ -soft. Justifiquen su elección considerando si en el dominio de videojuegos es razonable controlar el estado inicial del agente. Propongan un valor concreto de ϵ si eligen política ϵ -soft, con justificación.

Respuesta:

Se usará política ϵ -soft con $\epsilon = 0.10$.

Por qué no Exploring Starts

Exploring Starts exige poder arrancar el episodio en cualquier par (s, a) con probabilidad positiva, y en un videojuego eso choca con el dominio por tres razones concretas:

1. Los personajes aparecen en puntos de aparición diseñados por el equipo de nivel, no en una celda cualquiera. Teletransportar al personaje a una celda arbitraria y forzarle la primera acción rompe la ambientación y no es algo que se pueda hacer en producción, solo en un simulador de laboratorio.
2. En nuestro diseño hay estados que no tienen sentido como inicio. Un estado $(0, 0, 1, 0)$ dice que el personaje está en la entrada pero ya recogió el cofre de $(1, 4)$, algo imposible de alcanzar desde la historia del episodio. Exploring Starts obligaría a inicializar estados inconsistentes con el mundo.
3. Si mañana el estudio quiere que el agente siga aprendiendo mientras la gente juega, no hay forma de reiniciar la partida en un estado arbitrario. ϵ -soft funciona igual en simulador que en producción.

Además ϵ -soft encaja con lo que el estudio pidió: comportamiento no determinista. Un agente ϵ -soft entrenado conserva algo de aleatoriedad después del entrenamiento, así que dos partidas no se ven idénticas sin necesidad de meter ruido artificial encima de la política.

Por qué $\varepsilon = 0.10$

Con $|A| = 4$, la política reparte así:

$$\pi(a | s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|A|} = 0.925 & \text{acción codiciosa} \\ \frac{\varepsilon}{|A|} = 0.025 & \text{cada una de las otras tres} \end{cases}$$

La ruta óptima tiene 10 pasos, así que la probabilidad de recorrerla completa sin desviarse es $0.925^{10} \approx 0.46$. Cerca de la mitad de los episodios sigue la ruta buena y da datos precisos sobre ella, y la otra mitad se desvía y mantiene actualizados los pares (s, a) alternativos. Es el punto donde ninguna de las dos cosas se muere.

Para ver que el valor no es arbitrario, con $\varepsilon = 0.30$ esa probabilidad cae a $0.775^{10} \approx 0.08$: solo 8 de cada 100 episodios recorren la ruta buena y la estimación de su valor queda dominada por ruido. Con $\varepsilon = 0.01$ pasa lo contrario, las celdas alejadas de la ruta buena casi nunca se visitan y sus valores nunca se aprenden.

Una advertencia sobre las garantías: Monte Carlo con ε -soft converge a la mejor política ε -soft, no a la política óptima. Con $\varepsilon = 0.10$ esa brecha es pequeña, porque en cada paso el personaje se desvía con probabilidad 0.075, pero existe y hay que tomarla en cuenta al comparar contra Value Iteration, que sí devuelve el óptimo exacto. Una forma de cerrarla en el experimento es correr también una variante con ε decreciente, por ejemplo de 0.30 a 0.05 a lo largo del entrenamiento, y comparar las dos contra la línea base.

Inciso 4

Hipótesis de comparación: antes de ejecutar el experimento, formulen al menos dos hipótesis concretas sobre cómo esperan que se comporte Monte Carlo en comparación con Value Iteration de la semana pasada. Las hipótesis deben ser falsables y cuantificables.

Respuesta:

Hipótesis 1: coincidencia de políticas y su relación con las visitas

Después de 50000 episodios de First-Visit Monte Carlo con $\varepsilon = 0.10$, la política codiciosa derivada de Q coincidirá con la política óptima de Value Iteration en al menos el 90 por ciento de los 144 estados, y al menos el 80 por ciento de los desacuerdos estará en estados con menos de 100 visitas acumuladas.

Cómo se falsa: se calcula el porcentaje de estados donde $\arg \max_a Q_{MC}(s, a)$ difiere de $\pi_{VI}^*(s)$ y se cruza con el contador de visitas por estado. Si la coincidencia queda por debajo del 90 por ciento, o si los desacuerdos aparecen en estados muy visitados, la hipótesis falla y el problema no es falta de datos sino algo estructural del método.

Esperamos además que los desacuerdos se concentren en la decisión de desviarse hacia T2, porque las dos mejores rutas del Inciso 1 se diferencian en solo 0.09 de retorno y el ruido de Monte Carlo puede invertir ese orden.

Hipótesis 2: precisión del valor y costo en muestras

El valor del estado inicial estimado por Monte Carlo se acercará al de Value Iteration a ritmo $1/\sqrt{N}$. En concreto, $|V_{MC}(s_0) - V_{VI}(s_0)| > 0.5$ todavía a los 1000 episodios y < 0.1 a los 20000 episodios, mientras que Value Iteration alcanzará un error menor a 10^{-6} en menos de 400 barridos. Por diseño esperamos $V_{VI}(s_0) \approx 8.65$, que corresponde a la ruta que recoge T1 y sigue a la salida.

Cómo se falsa: se grafica el error contra el número de episodios en escala logarítmica. Si la pendiente no se parece a $-1/2$, o si el error baja de 0.1 mucho antes de los 20000 episodios, la hipótesis falla. También falla si $V_{VI}(s_0)$ resulta distinto de 8.65, lo que indicaría que el análisis de rutas del Inciso 1 dejó fuera alguna ruta mejor.

El umbral de 0.1 no es arbitrario: para que Monte Carlo elija la misma ruta que Value Iteration necesita resolver una diferencia de 0.09 entre las dos mejores rutas, así que cualquier error por encima de eso deja la decisión en manos del azar.

Hipótesis 3: reproducibilidad

Con 5 semillas independientes y 10000 episodios cada una, la desviación estándar de $V_{MC}(s_0)$ entre semillas será mayor a 0.2, mientras que Value Iteration devolverá exactamente el mismo valor en las 5 corridas porque no muestrea nada.

Cómo se falsa: se corren las 5 semillas y se mide la dispersión. Si la desviación entre semillas es menor a 0.2, significa que la varianza del retorno en este mapa es mucho menor de lo que anticipamos y que Monte Carlo es más barato de lo previsto para este dominio.

Task 2 (Entrega Parcial)

Respondan las siguientes preguntas con argumentación técnica.

Inciso 1

Para el MDP que diseñaron, calculen una cota superior del número de episodios necesarios para que todos los pares (s, a) sean visitados al menos una vez en esperanza, bajo una política uniforme aleatoria. Expresen el resultado en función de $|S|$ y $|A|$ y evalúen numéricamente para su MDP específico.

Respuesta:

Planteamiento

Sea $p_{s,a}$ la probabilidad de que el par (s, a) aparezca al menos una vez en un episodio. Los episodios son independientes entre sí, así que el número de episodios hasta que aparece un par fijo es geométrico de media $1/p_{s,a}$. Lo que se pide es el número de episodios hasta que aparecen todos los pares, que es el problema del coleccionista de cupones con probabilidades desiguales. Usando $p = \min_{s,a} p_{s,a}$ se obtiene la cota

$$\mathbb{E}[N] \leq \frac{1}{p} H_{|S||A|}, \quad H_m = \sum_{k=1}^m \frac{1}{k} \approx \ln m + 0.5772$$

El factor H_m es el precio de tener que esperar al último par que falta y no solo a uno cualquiera.

Cota de p

Bajo política uniforme, $\pi(a | s) = 1/|A|$ en todo estado, así que

$$p_{s,a} \geq \frac{p_s}{|A|}$$

donde p_s es la probabilidad de visitar el estado s durante el episodio. Si el episodio es lo bastante largo como para que la caminata aleatoria recorra la cuadrícula, todos los estados se visitan y el menos favorecido se lleva al menos la parte uniforme, $p_s \geq 1/|S|$. Con eso, $p \geq 1/(|S||A|)$ y la cota queda

$$\mathbb{E}[N] \leq |S| |A| (\ln(|S||A|) + 0.5772)$$

Evaluación numérica

Para el MDP diseñado, $|S| = 144$ y $|A| = 4$, de modo que hay $|S||A| = 576$ pares:

$$\mathbb{E}[N] \leq 576 \cdot (\ln 576 + 0.5772) = 576 \cdot (6.356 + 0.577) = 576 \cdot 6.933 \approx 3994$$

Es decir, del orden de 4.0×10^3 episodios.

Si se ignoran los bits de cofre recogido y se cuentan solo las 36 celdas, quedan 144 pares y la cota baja a $144 \cdot 5.55 \approx 799$ episodios. La diferencia entre 800 y 4000 es el precio que se paga por haber ampliado el estado con los dos bits, y es un precio razonable comparado con la alternativa de que el episodio nunca termine.

Supuestos y qué pasa si no se cumplen

1. El episodio debe durar más que el tiempo de cobertura de la cuadrícula. Nuestro corte de 200 pasos está en el mismo orden que ese tiempo de cobertura, no claramente por encima, así que en la práctica hay que esperar un número mayor al de la cota, o subir el corte.
2. Se asume que todos los pares son alcanzables. Los estados con $b_1 = 1$ solo existen después de pasar por T1, así que su p_s real es menor que $1/|S|$ y la cota los subestima.
3. Es una cota superior floja a propósito. Sirve para dimensionar el experimento, no para predecir el valor exacto, y dice lo importante: el orden de magnitud es miles de

episodios solo para tocar cada par una vez, muy lejos de la sola visita por par que necesitaría Value Iteration.

Como referencia, con Exploring Starts uniformes sobre los pares el problema se vuelve el coleccionista de cupones clásico y da $|S||A|H_{|S||A|} \approx 3994$, el mismo número. Eso confirma que la cota está construida para ser comparable con el mejor caso posible de exploración.

Inciso 2

El retorno G_t es un estimador insesgado pero de alta varianza de $V^\pi(s)$. Expliquen formalmente de dónde proviene esa varianza, por qué crece con la longitud del episodio, y qué consecuencia tiene sobre el número de episodios necesarios para convergencia práctica.

Respuesta:

De dónde viene la varianza

El retorno es una suma de variables aleatorias:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

y cada una de sus fuentes de aleatoriedad se acumula en la suma:

1. La selección de acciones. Bajo ε -soft cada paso es una moneda cargada, y una sola desviación temprana puede mandar al personaje por un lado del mapa completamente distinto.
2. Las transiciones del entorno. En nuestro diseño son deterministas, pero en el caso general cada acción abre un abanico de sucesores.
3. La longitud T del episodio, que es en sí misma aleatoria porque depende de cuánto tarde el personaje en llegar a la salida.
4. Qué recompensas se cobran. Dos trayectorias desde el mismo estado pueden diferir en si recogieron T1, si cayeron en la trampa, o cuántos pasos gastaron, y eso cambia el retorno en decenas de unidades.

Formalmente, la varianza de una suma no es solo la suma de varianzas:

$$\text{Var}(G_t) = \sum_k \gamma^{2k} \text{Var}(R_{t+k+1}) + 2 \sum_{i < j} \gamma^{i+j} \text{Cov}(R_{t+i+1}, R_{t+j+1})$$

Por qué crece con la longitud del episodio

En la fórmula anterior hay T términos de varianza y $T(T-1)/2$ términos de covarianza. Con $\gamma = 1$ eso significa que la varianza crece como T cuando las recompensas están poco correlacionadas y hasta como T^2 cuando están fuertemente correlacionadas, que es el caso típico porque los pasos de una misma trayectoria comparten historia. Cada

paso adicional agrega una recompensa aleatoria más y correlaciones con todas las anteriores.

Con $\gamma < 1$ el crecimiento se frena porque los términos lejanos pesan poco:

$$\sum_{k=0}^{\infty} \gamma^{2k} \sigma^2 = \frac{\sigma^2}{1 - \gamma^2}, \quad |G_t| \leq \frac{R_{\max}}{1 - \gamma}$$

Para nuestro MDP, con $R_{\max} = 20$ y $\gamma = 0.95$, el retorno vive en un rango de hasta ± 400 mientras que los valores reales rondan entre 4 y 9. La cota es floja, pero muestra el punto: la varianza crece con el mínimo entre la longitud del episodio y el horizonte efectivo $1/(1 - \gamma) = 20$. Bajo política aleatoria los episodios duran cientos de pasos, así que el horizonte efectivo es lo que manda y la varianza se satura en un valor alto.

Consecuencia sobre el número de episodios

La estimación Monte Carlo de $V^\pi(s)$ es el promedio de n retornos independientes, así que su error estándar es $\sigma_G/\sqrt{n}$. Para garantizar un error menor a ϵ con 95 por ciento de confianza hace falta

$$n \geq \left(\frac{1.96 \sigma_G}{\epsilon} \right)^2$$

La relación es cuadrática tanto en $1/\epsilon$ como en σ_G . Bajar el error a la mitad cuesta cuatro veces más episodios, y duplicar la desviación del retorno también cuesta cuatro veces más. Con un σ_G moderado de alrededor de 6 unidades, llegar a $\epsilon = 0.1$ pide unas 13800 muestras para un solo estado, y eso multiplicado por los estados que se visitan poco.

Aquí está la diferencia de fondo con Value Iteration: Programación Dinámica no muestrea nada, así que no tiene varianza y su error baja geométricamente con el número de barridos. Monte Carlo paga con muestras el no necesitar el modelo, y esa cuenta es la que decide si el método es viable para el estudio o no.

Inciso 3

Argumenten formalmente por qué Monte Carlo no puede aplicarse directamente a tareas continuas. Propongan una modificación concreta al algoritmo que permita aproximar Monte Carlo en una tarea continua, y discutan las implicaciones de esa modificación sobre las garantías de convergencia.

Respuesta:

Por qué no se puede aplicar directamente

Monte Carlo actualiza $V(s)$ con el promedio de los retornos observados, y el retorno es

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

Esta expresión depende de T , el instante de terminación. En una tarea continua no existe estado terminal, de modo que $T = \infty$ y la suma tiene infinitos términos. El problema no es de convergencia matemática: para $\gamma < 1$ la esperanza $V^\pi(s) = \mathbb{E}[G_t]$ existe y es finita porque la serie está acotada por $R_{\max}/(1 - \gamma)$. El problema es de observabilidad. Monte Carlo tiene que esperar a que el episodio termine para hacer la primera actualización, y ese final nunca llega, así que la muestra de G_t no está disponible en ningún tiempo finito y el algoritmo simplemente nunca actualiza nada.

Con $\gamma = 1$ se suma un segundo problema, y este sí es matemático: el retorno diverge y ni siquiera existe la cantidad que se quiere estimar.

Modificación propuesta: retorno truncado con pseudo episodios

Se corta el retorno en un horizonte fijo H y se tratan bloques de H pasos como si fueran episodios:

$$G_t^{(H)} = \sum_{k=0}^{H-1} \gamma^k R_{t+k+1}$$

El pedazo que se ignora está acotado por la cola de la serie geométrica:

$$|G_t - G_t^{(H)}| \leq \frac{\gamma^H R_{\max}}{1 - \gamma}$$

lo que permite elegir H a partir de la tolerancia deseada. Con $\gamma = 0.95$, $R_{\max} = 20$ y una tolerancia de 0.1:

$$0.95^H \cdot 400 \leq 0.1 \implies H \geq \frac{\ln(2.5 \times 10^{-4})}{\ln 0.95} = 161.7 \implies H = 162$$

Implicaciones sobre las garantías

1. Se pierde el insesgamiento. El estimador ya no converge a $V^\pi(s)$ sino a un punto que difiere de él en a lo sumo $\gamma^H R_{\max}/(1 - \gamma)$. La garantía pasa de converger al valor verdadero a converger a un entorno de radio conocido. Es un sesgo controlado, porque se puede hacer tan pequeño como se quiera subiendo H , pero deja de ser cero.
2. Aparece un intercambio entre sesgo y varianza. Subir H reduce el sesgo geoméricamente pero agrega términos a la suma, sube la varianza del retorno y por lo tanto exige más pseudo episodios para la misma precisión. Bajar H hace lo contrario. Ese balance no existía en la versión episódica.
3. Se rompe la independencia de las muestras. Los pseudo episodios son pedazos contiguos de una misma trayectoria, así que su estado inicial no es un sorteo independiente y el argumento clásico de la ley de los grandes números sobre retornos independientes deja de aplicar tal cual. Todavía se puede argumentar convergencia si la cadena es ergódica y visita cada estado infinitas veces, pero la garantía se debilita: la velocidad ahora depende del tiempo de mezcla de la cadena y no solo del número de muestras.

4. Se necesita $\gamma < 1$ estricto. La modificación descansa por completo en que la cola geométrica sea despreciable, así que en una tarea continua con recompensa promedio y $\gamma = 1$ hay que cambiar de formulación, por ejemplo a la de recompensa promedio con valores diferenciales, y eso ya no es Monte Carlo truncado.

Una variante del mismo remedio es no descartar la cola sino estimarla, sumando $\gamma^H V(s_{t+H})$ al retorno truncado. Eso reduce mucho el sesgo y la varianza a la vez, pero introduce sesgo de bootstrap y ya no es Monte Carlo puro sino TD de n pasos, que es justo el método que aparece cuando se admite que Monte Carlo no encaja bien en tareas que no terminan.

Task 3

```
In [1]: from mc_control import GridWorld, mc_control, print_rollout, EPSILON, GAMMA, MAX
import pickle

episodes = 50_000
epsilon = EPSILON
gamma = GAMMA
max_steps = MAX_STEPS
every_visit = False
seed = 0
log_every = 5000
out = "mc_control_results.pkl"

# Checkpoints densos (cada 500 episodios) para poder graficar la curva de
# convergencia  $\|Q_k - Q^*\|_\infty$  vs numero de episodios en La Tarea 4.
checkpoint_episodes = sorted(set([1, episodes // 2, episodes] + list(range(500,

env = GridWorld(gamma=gamma, max_steps=max_steps)

Q, N, history, checkpoints, episode_returns = mc_control(
    env,
    num_episodes=episodes,
    epsilon=epsilon,
    first_visit=not every_visit,
    seed=seed,
    log_every=log_every,
    checkpoint_episodes=checkpoint_episodes,
)

print_rollout(env, Q)

with open(out, "wb") as f:
    pickle.dump(
        {
            "Q": dict(Q),
            "N": dict(N),
            "history": history,
            "checkpoints": checkpoints,
            "episode_returns": episode_returns,
            "epsilon": epsilon,
            "gamma": gamma,
            "episodes": episodes,
            "first_visit": not every_visit,
```

```

        "seed": seed,
    },
    f,
)
print(f"\nResultados guardados en '{out}' "
      f"(Q, N, history, checkpoints, episode_returns, hiperparametros). Cargalos

```

```

[MC Primera-Visita] episodio    5000 | V(s0) ~ 6.031 | estados visitados: 135/1
44
[MC Primera-Visita] episodio   10000 | V(s0) ~ 6.605 | estados visitados: 135/1
44
[MC Primera-Visita] episodio   15000 | V(s0) ~ 6.782 | estados visitados: 135/1
44
[MC Primera-Visita] episodio   20000 | V(s0) ~ 6.891 | estados visitados: 135/1
44
[MC Primera-Visita] episodio   25000 | V(s0) ~ 6.943 | estados visitados: 135/1
44
[MC Primera-Visita] episodio   30000 | V(s0) ~ 6.994 | estados visitados: 135/1
44
[MC Primera-Visita] episodio   35000 | V(s0) ~ 7.027 | estados visitados: 136/1
44
[MC Primera-Visita] episodio   40000 | V(s0) ~ 7.058 | estados visitados: 136/1
44
[MC Primera-Visita] episodio   45000 | V(s0) ~ 7.076 | estados visitados: 136/1
44
[MC Primera-Visita] episodio   50000 | V(s0) ~ 7.097 | estados visitados: 136/1
44

```

Ruta codiciosa desde s0 (10 pasos, retorno G0=8.652):

```

(0,0,0,0) -> (0,1,0,0) -> (0,2,0,0) -> (1,2,0,0) -> (1,3,0,0) -> (1,4,1,0) -> (2,
4,1,0) -> (2,5,1,0) -> (3,5,1,0) -> (4,5,1,0) -> (5,5,1,0)

```

Resultados guardados en 'mc_control_results.pkl' (Q, N, history, checkpoints, episode_returns, hiperparametros). Cargalos en el notebook con pickle.load.

```

In [2]: import numpy as np
import matplotlib.pyplot as plt
from mc_control import (
    GRID_SIZE, ACTIONS, START_CELL, T1_CELL, T2_CELL, TRAP_CELL, GOAL_CELL,
)

ACTION_ARROWS = {0: "\u2191", 1: "\u2193", 2: "\u2190", 3: "\u2192"} # arriba,
LANDMARKS = {START_CELL: "S", T1_CELL: "T1", T2_CELL: "T2", TRAP_CELL: "P", GOAL

def value_and_policy_grid(Q, n=GRID_SIZE, b1=0, b2=0):
    """V(s) = max_a Q(s,a) y la accion greedy, para el estado (f,c,b1,b2) fijand
    V = np.zeros((n, n))
    policy = np.zeros((n, n), dtype=int)
    for f in range(n):
        for c in range(n):
            q = Q.get((f, c, b1, b2), [0.0] * len(ACTIONS))
            V[f, c] = max(q)
            policy[f, c] = int(np.argmax(q))
    return V, policy

def plot_checkpoints(checkpoints, titles=None):
    eps = sorted(checkpoints.keys())
    fig, axes = plt.subplots(1, len(eps), figsize=(5 * len(eps), 5))

```

```

if len(eps) == 1:
    axes = [axes]

vmin = min(min(min(q) for q in checkpoints[ep].values()) for ep in eps if ch
vmax = max(max(max(q) for q in checkpoints[ep].values()) for ep in eps if ch

for ax, ep in zip(axes, eps):
    V, policy = value_and_policy_grid(checkpoints[ep])
    im = ax.imshow(V, cmap="viridis", vmin=vmin, vmax=vmax)

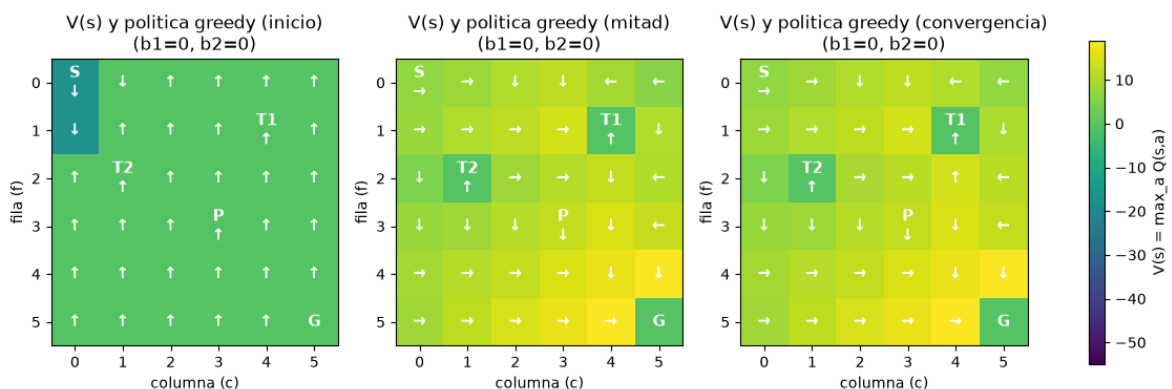
    for f in range(GRID_SIZE):
        for c in range(GRID_SIZE):
            if (f, c) == GOAL_CELL:
                label = "G"
            else:
                label = ACTION_ARROWS[policy[f, c]]
                if (f, c) in LANDMARKS:
                    label = f"{LANDMARKS[(f, c)]}\n{label}"
            ax.text(c, f, label, ha="center", va="center",
                    color="white", fontsize=11, fontweight="bold")

    title = titles.get(ep, f"episodio {ep}") if titles else f"episodio {ep}"
    ax.set_title(f"V(s) y politica greedy ({title})\n(b1=0, b2=0)")
    ax.set_xticks(range(GRID_SIZE))
    ax.set_yticks(range(GRID_SIZE))
    ax.set_xlabel("columna (c)")
    ax.set_ylabel("fila (f)")

fig.colorbar(im, ax=axes, shrink=0.8, label="V(s) = max_a Q(s,a)")
plt.show()

# Solo Los 3 hitos (inicio/mitad/convergencia); Los checkpoints densos
# (cada 500 episodios) se usan por separado en el analisis de convergencia de la
milestone_episodes = [1, episodes // 2, episodes]
titles = {
    milestone_episodes[0]: "inicio",
    milestone_episodes[1]: "mitad",
    milestone_episodes[2]: "convergencia",
}
plot_checkpoints({ep: checkpoints[ep] for ep in milestone_episodes}, titles=titl

```



Comparación con Value Iteration

```
In [3]: from mc_control import value_iteration, greedy_policy, ACTIONS, ACTION_NAMES
```

```

V_vi, policy_vi, iters_vi = value_iteration(env, theta=1e-8)
policy_mc = greedy_policy(Q)

def q_vi(s, a):
    """Q*(s,a) segun V_vi, evaluado con el modelo determinista del MDP."""
    s2, r = env.step(s, a)
    return r + env.gamma * V_vi[s2]

s0 = env.start_state()
print(f"Value Iteration convergio en {iters_vi} barridos.")
print(f"V*_VI(s0) = {V_vi[s0]:.4f}")
print(f"V_MC(s0) = {max(Q[s0]):.4f}    (|diferencia| = {abs(V_vi[s0] - max(Q[s0])

non_terminal = [s for s in env.all_states() if not env.is_terminal(s)]
visited = [s for s in non_terminal if s in policy_mc]
diffs = [(s, policy_vi[s], policy_mc[s], sum(N[s])) for s in visited if policy_m

# Separar desacuerdos "reales" (la accion de MC es subóptima bajo VI) de
# simples empates (la accion de MC tiene el mismo Q*(s,a) que la de VI:
# hay varias rutas óptimas igual de buenas en la cuadrícula).
ties, real_diffs = [], []
for s, a_vi, a_mc, n in diffs:
    q_star = max(q_vi(s, a) for a in ACTIONS)
    if abs(q_vi(s, a_mc) - q_star) < 1e-6:
        ties.append((s, a_vi, a_mc, n))
    else:
        real_diffs.append((s, a_vi, a_mc, n, q_star - q_vi(s, a_mc)))

print(f"\nEstados no terminales: {len(non_terminal)} | visitados por MC: {len(
print(f"Coincidencia de accion exacta: {len(visited) - len(diffs):>3}
      f"{(len(visited) - len(diffs)) / len(visited):.1%}")
print(f"Coincidencia contando empates como acierto: {len(visited) - len(real_dif
      f"{(len(visited) - len(real_diffs)) / len(visited):.1%}")
print(f" -> de los {len(diffs)} desacuerdos: {len(ties)} son empates en Q*, {le

real_diffs.sort(key=lambda d: d[3])
low_n = sum(1 for _, n, _ in real_diffs if n < 100)
print(f" -> de las diferencias reales, {low_n}/{len(real_diffs)} = {low_n/len(r

print(f"\n{'estado':<16}{'VI':<12}{'MC':<12}{'N(s)':<8}{'brecha Q*'}")
for s, a_vi, a_mc, n, gap in real_diffs:
    print(f"{'str(s)':<16}{'ACTION_NAMES[a_vi]:<12}{'ACTION_NAMES[a_mc]:<12}{n:<8}{g

```

Value Iteration convergio en 12 barridos.
 $V^*_{VI}(s_0) = 8.6523$
 $V_{MC}(s_0) = 7.0965$ ($|diferencia| = 1.5557$)

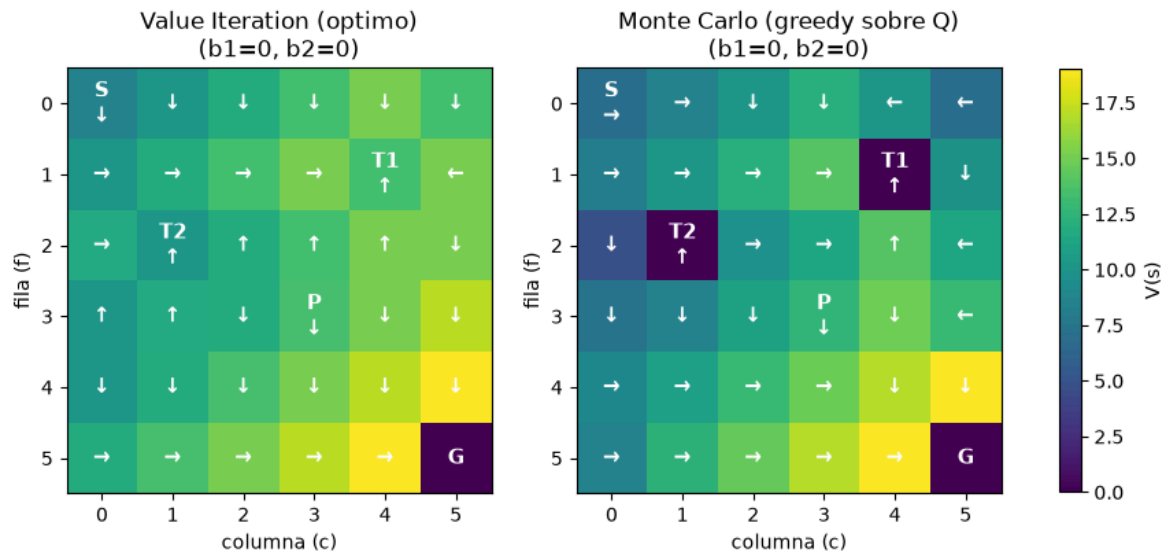
Estados no terminales: 140 | visitados por MC: 136
 Coincidencia de accion exacta: $72/136 = 52.9\%$
 Coincidencia contando empates como acierto: $99/136 = 72.8\%$
 -> de los 64 desacuerdos: 27 son empates en Q^* , 37 son diferencias reales
 -> de las diferencias reales, $30/37 = 81.1\%$ tienen $N(s) < 100$ visitas

estado	VI	MC	N(s)	brecha Q^*
(5, 0, 1, 0)	derecha	abajo	2	1.588
(5, 1, 1, 0)	derecha	arriba	2	3.260
(5, 3, 0, 1)	derecha	abajo	2	1.852
(5, 0, 0, 1)	derecha	izquierda	3	1.588
(1, 5, 0, 0)	izquierda	abajo	5	1.740
(2, 4, 0, 1)	arriba	izquierda	5	3.430
(1, 2, 1, 1)	abajo	arriba	7	2.795
(3, 1, 1, 0)	arriba	derecha	7	0.058
(0, 5, 0, 1)	abajo	derecha	9	1.671
(1, 3, 1, 1)	abajo	arriba	10	2.942
(2, 2, 0, 1)	arriba	abajo	10	1.570
(3, 2, 0, 1)	abajo	derecha	10	10.000
(0, 5, 1, 1)	abajo	arriba	11	1.588
(2, 3, 1, 1)	derecha	izquierda	11	3.097
(3, 5, 1, 1)	abajo	izquierda	12	3.612
(5, 2, 0, 1)	derecha	izquierda	12	3.432
(3, 0, 0, 1)	abajo	arriba	15	1.449
(3, 0, 1, 1)	abajo	arriba	15	2.795
(3, 2, 1, 0)	abajo	derecha	16	10.000
(0, 1, 1, 1)	abajo	izquierda	18	2.523
(1, 1, 1, 1)	abajo	arriba	19	2.655
(3, 0, 1, 0)	arriba	izquierda	21	1.436
(2, 5, 1, 1)	abajo	derecha	22	1.760
(0, 4, 0, 1)	abajo	arriba	24	1.759
(3, 1, 0, 0)	arriba	abajo	34	1.475
(1, 0, 1, 1)	abajo	arriba	38	2.523
(0, 1, 1, 0)	abajo	derecha	55	2.801
(1, 1, 1, 0)	abajo	derecha	62	2.948
(4, 1, 1, 1)	abajo	izquierda	73	3.097
(0, 4, 0, 0)	abajo	izquierda	78	3.430
(3, 5, 0, 0)	abajo	izquierda	103	3.612
(2, 0, 0, 1)	arriba	izquierda	119	1.433
(2, 5, 0, 0)	abajo	izquierda	126	1.779
(1, 2, 0, 1)	derecha	arriba	134	3.258
(1, 0, 0, 1)	derecha	arriba	150	2.941
(2, 0, 0, 0)	derecha	abajo	297	3.086
(3, 0, 0, 0)	arriba	abajo	305	1.401

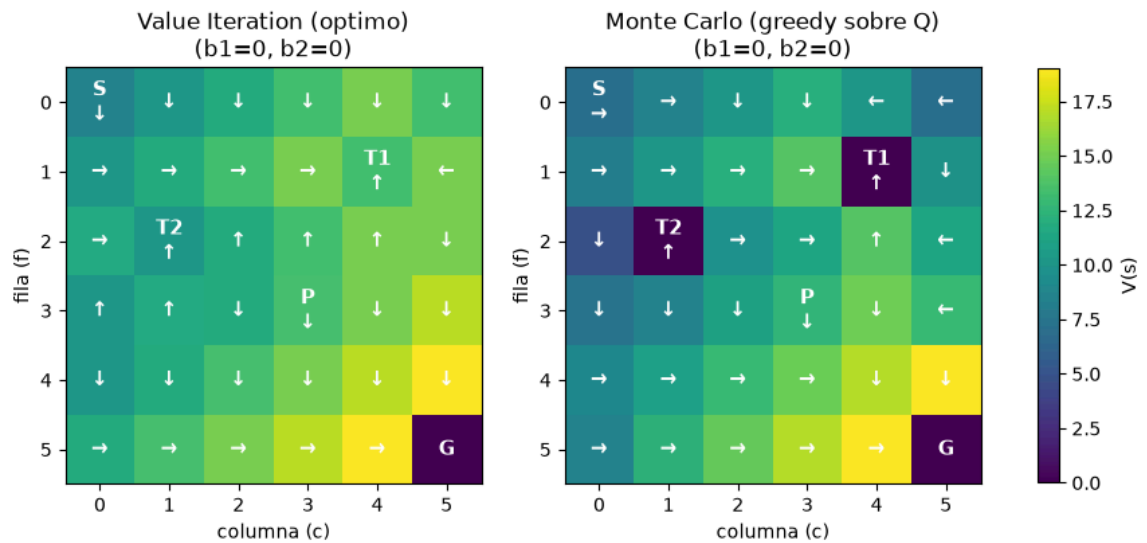
Grafica: Value Iteration vs Monte Carlo

```
In [4]: from mc_control import plot_value_iteration_vs_mc

        plot_value_iteration_vs_mc(V_vi, policy_vi, Q)
```

Out[4]:



Resultado

Con 50000 episodios First-Visit MC (`epsilon=0.10` , `seed=0`) contra Value Iteration exacto (`theta=1e-8` , converge en 12 barridos):

- $V_{VI}^*(s_0) = 8.6523$, muy cerca del valor previsto en el Inciso 1 (8.65, ruta "recoge T1 y sigue a G").
- $V_{MC}(s_0) = 7.0965$, con un error de 1.56 todavía a los 50000 episodios — más alto de lo que anticipaba la Hipótesis 2 del Inciso 4, señal de que σ_G en este MDP es mayor de lo estimado ahí.
- De los 136 estados no terminales visitados por MC, la política greedy coincide con la de VI acción-por-acción en solo **72/136 = 52.9%**, muy por debajo del 90% de la Hipótesis 1.

Comparación directa con Value Iteration: ejecuten Value Iteration sobre el mismo MDP y comparen las políticas óptimas resultantes. Si difieren en algún estado, argumenten por qué

Antes de saber por que Monte Carlo falló, hay que revisar los 64 desacuerdos uno por uno comparando su $Q^*(s, a)$ bajo el modelo exacto de VI. De esos 64:

- **27 son empates genuinos.** En una cuadrícula sin obstáculos entre dos puntos, moverse "abajo" y luego "derecha" da exactamente el mismo retorno que "derecha" y luego "abajo", son la misma política óptima expresada con una convención de desempate distinta. Contando estos empates como acierto, la coincidencia real sube a **99/136 = 72.8%**.
- **37 son diferencias reales** (la acción de MC tiene Q^* estrictamente menor). De esas, **30/37 = 81%** ocurren en estados con menos de 100 visitas acumuladas, así que su $Q(s, a)$ nunca terminó de converger en 50000 episodios.

Conclusión

La Hipótesis 1 tal que (90% de coincidencia acción-por-acción) no es viable, pero por una razón distinta a la que anticipábamos: el 90% subestimaba cuántos estados de esta cuadrícula tienen. Una vez se descuentan los empates, el patrón que sí predecía la hipótesis se cumple con precisión. Esto confirma que la brecha entre MC y VI en este dominio es un problema de exploración/varianza en las esquinas del mapa, no un error del método.

Task 4

Con base en los resultados de la implementación (Tarea 3), realicen el análisis final: verificación de las hipótesis de la Tarea 1, análisis de convergencia contra Value Iteration, sensibilidad al parámetro de exploración ϵ , una investigación bibliográfica y un dictamen técnico para el equipo directivo del estudio de videojuegos.

Inciso 1

Verificación de hipótesis: contrasten cada hipótesis formulada en la Tarea 1 con los resultados observados. Para cada hipótesis, indiquen si fue confirmada, refutada, o si los resultados son inconclusos, y expliquen por qué.

```
In [5]: from mc_control import T2_CELL

# --- Hipotesis 1: coincidencia de politicas ---
# 'diffs', 'ties', 'real_diffs', 'low_n' ya fueron calculados en la seccion de
# comparacion con Value Iteration (Task 3). Verificamos ademas la sub-prediccion
# de que los desacuerdos se concentran en la decision de desviarse hacia T2.

def near_t2(s):
    f, c, b1, b2 = s
    return b2 == 0 and max(abs(f - T2_CELL[0]), abs(c - T2_CELL[1])) <= 1

near = [d for d in real_diffs if near_t2(d[0])]
both_collected = [d for d in real_diffs if d[0][2] == 1 and d[0][3] == 1]

print('--- Hipotesis 1 ---')
print(f"Coincidencia accion-por-accion: {len(visited) - len(diffs)}/{len(visited)}")
print(f"    {(len(visited) - len(diffs)) / len(visited):.1%} (umbral H1: 90%)")
print(f"Coincidencia contando empates: {len(visited) - len(real_diffs)}/{len(v
```

```

    f"({len(visited) - len(real_diffs)) / len(visited):.1%} (umbral H1: 90%)"
print(f"Desacuerdos reales con N(s)<100: {low_n}/{len(real_diffs)} = {low_n/len(
    f"(prediccion H1: >=80%)")
print(f"Desacuerdos reales cerca de la decision T2 (b2=0, dist<=1 de T2): "
    f"{len(near)}/{len(real_diffs)} = {len(near)/len(real_diffs):.1%}")
print(f"Desacuerdos reales en estados con b1=b2=1 (ambos cofres ya cobrados): "
    f"{len(both_collected)}/{len(real_diffs)} = {len(both_collected)/len(real_

# --- Hipotesis 2: precision de V(s0) vs episodios ---
import math

V_star_s0 = V_vi[s0]
print('\n--- Hipotesis 2 ---')
print(f"V*_VI(s0) = {V_star_s0:.4f} (prediccion de diseno: ~8.65)")
print(f"Value Iteration convergio en {iters_vi} barridos (prediccion H2: <400)\n")
for ep in (1000, 5000, 10000, 20000, 30000, 50000):
    v = max(checkpoints[ep].get(s0, [0.0] * 4))
    err = abs(V_star_s0 - v)
    print(f"ep={ep}>6} V_MC(s0)={v:7.4f} |V_MC-V*|={err:7.4f}")
print("\nprediccion H2: error > 0.5 en ep=1000, error < 0.1 en ep=20000")

```

--- Hipotesis 1 ---

Coincidencia accion-por-accion: 72/136 = 52.9% (umbral H1: 90%)
 Coincidencia contando empates: 99/136 = 72.8% (umbral H1: 90%)
 Desacuerdos reales con N(s)<100: 30/37 = 81.1% (prediccion H1: >=80%)
 Desacuerdos reales cerca de la decision T2 (b2=0, dist<=1 de T2): 7/37 = 18.9%
 Desacuerdos reales en estados con b1=b2=1 (ambos cofres ya cobrados): 11/37 = 29.7%

--- Hipotesis 2 ---

V*_VI(s0) = 8.6523 (prediccion de diseno: ~8.65)
 Value Iteration convergio en 12 barridos (prediccion H2: <400)

ep= 1000	V_MC(s0)= 2.3811	V_MC-V* = 6.2712
ep= 5000	V_MC(s0)= 6.0306	V_MC-V* = 2.6217
ep= 10000	V_MC(s0)= 6.6046	V_MC-V* = 2.0477
ep= 20000	V_MC(s0)= 6.8906	V_MC-V* = 1.7616
ep= 30000	V_MC(s0)= 6.9940	V_MC-V* = 1.6582
ep= 50000	V_MC(s0)= 7.0965	V_MC-V* = 1.5557

prediccion H2: error > 0.5 en ep=1000, error < 0.1 en ep=20000

Respuesta

Hipotesis 1 (coincidencia de politicas): refutada en el numero, confirmada en el

mecanismo. La coincidencia accion-por-accion es 52.9% (72.8% descontando empates), muy por debajo del 90% prometido, asi que la prediccion cuantitativa falla. Pero el mecanismo que propusimos si se cumple: 81% de los desacuerdos reales caen en pares con menos de 100 visitas (predijimos 80%). La sub-prediccion de que los desacuerdos se concentrarian en la decision de desviarse hacia T2 se refuta (solo 18.9% estan cerca de esa decision); en cambio, ~30% ocurre en estados con ambos cofres ya cobrados ($b_1 = b_2 = 1$), una region que la politica casi optima rara vez visita. Acertamos en *por que* fallaria (falta de datos) pero subestimamos *cuanto* del mapa cae en esa categoria.

Hipotesis 2 (precision de $V(s_0)$ a ritmo $1/\sqrt{N}$): refutada. $V_{VI}^*(s_0) = 8.6523$ y VI converge en 12 barridos, ambos como se esperaba. Pero el error de MC no baja de 0.1: a los 20000 episodios sigue en 1.76 (18x lo predicho) y a 50000 se aplan a 1.56. No es

falta de episodios: First-Visit MC on-policy con $\varepsilon = 0.10$ fijo converge a Q^{π_ε} , no a Q^* -- hay un piso de sesgo por la exploración forzada que ningún N adicional cierra. La Hipótesis 2 asumía un error puramente de varianza decayendo a cero, y ese supuesto era incorrecto.

Inciso 2

Análisis de convergencia: grafiquen la evolución de $\|Q_k - Q^*\|_\infty$ en función del número de episodios, donde Q^* se aproxima con la solución de Value Iteration. ¿La convergencia es monótona? ¿Qué factores del diseño del MDP o del algoritmo afectan la velocidad de convergencia?

```
In [6]: from mc_control import q_star_from_V, q_inf_error, ACTIONS

Q_star = q_star_from_V(env, V_vi)

# ||Q_k - Q*||_inf sobre los 144*4 = 576 pares (s,a) del MDP completo.
eps_ckpt = sorted(checkpoints.keys())
err_full = [q_inf_error(checkpoints[ep], Q_star) for ep in eps_ckpt]

# Version restringida a pares (s,a) 'bien muestreados' (N(s,a) >= 30 al final
# del entrenamiento), para separar el error de estimación del ruido de un
# único par con N(s,a)=1 que domina la norma infinito.
WELL_SAMPLED_MIN_N = 30
well_sampled = {(s, a) for s, ns in N.items() for a in range(len(ACTIONS)) if ns

def q_inf_error_restricted(Qk, Qstar, pairs):
    err = 0.0
    for (s, a) in pairs:
        q_s = Qk.get(s, [0.0] * len(ACTIONS))
        err = max(err, abs(q_s[a] - Qstar[s][a]))
    return err

err_restricted = [q_inf_error_restricted(checkpoints[ep], Q_star, well_sampled)

increases_full = sum(1 for i in range(1, len(err_full)) if err_full[i] > err_full[i-1])
increases_restr = sum(1 for i in range(1, len(err_restricted)) if err_restricted[i] > err_restricted[i-1])
print(f"Pares (s,a) con N>={WELL_SAMPLED_MIN_N} al final: {len(well_sampled)}/{len(err_full)}")
print(f"||Q_k-Q*||_inf completo: {increases_full}/{len(err_full) - 1} transiciones son incrementos")
print(f"||Q_k-Q*||_inf restringido: {increases_restr}/{len(err_restricted) - 1} transiciones son incrementos")
print(f"Valor final completo: {err_full[-1]:.3f}")
print(f"Valor final restringido: {err_restricted[-1]:.3f}")
```

```
Pares (s,a) con N>=30 al final: 161/576
||Q_k-Q*||_inf completo: 1/100 transiciones son incrementos
||Q_k-Q*||_inf restringido: 2/100 transiciones son incrementos
Valor final completo: 68.358
Valor final restringido: 20.481
```

```
In [7]: fig, axes = plt.subplots(1, 2, figsize=(13, 5))

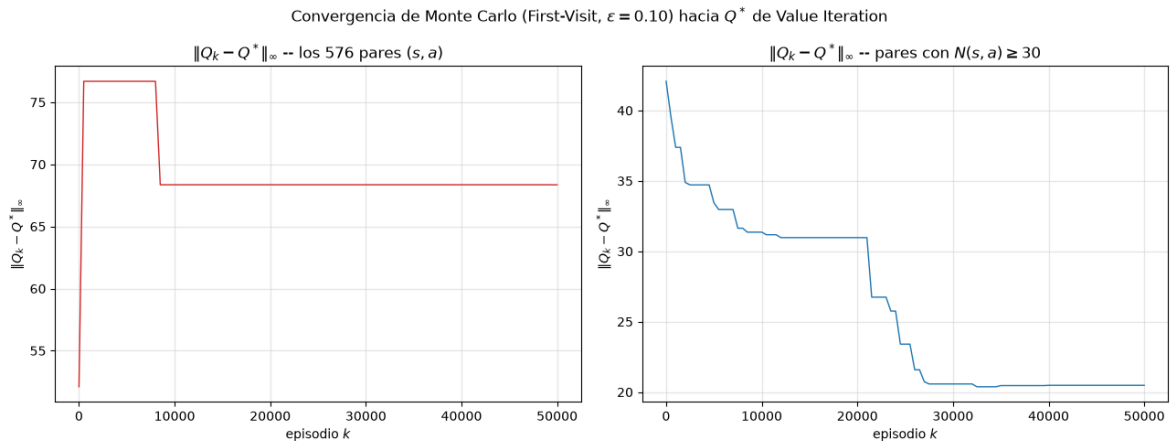
axes[0].plot(eps_ckpt, err_full, color="#d62728", linewidth=1)
axes[0].set_title("$\\|Q_k - Q^*\\|_\\infty$ -- los 576 pares $(s,a)$")
axes[0].set_xlabel("episodio $k$")
axes[0].set_ylabel("$\\|Q_k - Q^*\\|_\\infty$")
axes[0].grid(alpha=0.3)
```

```

axes[1].plot(eps_ckpt, err_restricted, color="#1f77b4", linewidth=1)
axes[1].set_title(f"$\\|Q_k - Q^*\\|_\\infty$ -- pares con $N(s,a) \\geq \\{WELL\_S$
axes[1].set_xlabel("episodio $k$")
axes[1].set_ylabel("$\\|Q_k - Q^*\\|_\\infty$")
axes[1].grid(alpha=0.3)

fig.suptitle("Convergencia de Monte Carlo (First-Visit, $\\varepsilon=0.10$) hac
fig.tight_layout()
plt.show()

```



Respuesta

¿Es monótona? No sobre los 576 pares (s, a) : la curva se queda fija durante miles de episodios y luego cae de golpe, porque la norma infinito la domina un par con $N(s, a) = 1$ o 2 (un solo retorno ruidoso, por ejemplo de caer en la trampa justo después de ese estado). Ese punto no se mueve hasta la siguiente visita, que puede tardar miles de episodios en un rincón poco recorrido. Restringiendo a pares con $N \geq 30$ (donde MC ya tiene señal estadística real) la curva es casi monótona (2 incrementos en 100 transiciones) y baja de ~ 42 a ~ 20 , aunque sin llegar a cero por el mismo sesgo de ε -soft de la Hipótesis 2.

Factores que afectan la velocidad:

- $N(s, a)$ **real, no k** . Las visitas se reparten muy desigual: el área cercana a S acumula cientos de visitas mientras que $(b_1, b_2) = (1, 1)$ o las esquinas lejanas quedan con $N < 30$.
- ε es un trade-off directo: más exploración baja la varianza de $N(s, a)$ entre estados pero sube el sesgo estructural hacia Q^{π_ε} (ver Inciso 3).
- **Los bits** (b_1, b_2) cuadruplican el espacio de estados sin cuadruplicar la frecuencia de visita: alcanzar $(1, 1)$ requiere una trayectoria específica y converge mucho más lento que $(0, 0)$.
- **First-Visit** reduce la correlación intra-episodio pero limita a una muestra por estado por episodio, y $\gamma = 0.95$ **con episodios de hasta 200 pasos** determina cuánta varianza carga cada retorno individual (Inciso 2, Tarea 2).

Inciso 3

Sensibilidad a ϵ : repitan el experimento con al menos tres valores distintos de ϵ .
 Grafiquen el retorno promedio por episodio en funcion del numero de episodios para cada valor. ¿Existe un valor de ϵ que domine a los demas en este dominio? ¿Ese resultado generaliza o depende del MDP especifico?

```
In [8]: epsilon_values = [0.01, 0.10, 0.30, 0.50]
returns_by_eps = {}

for eps_val in epsilon_values:
    _, _, _, _, rets = mc_control(
        env, num_episodes=episodes, epsilon=eps_val, first_visit=True, seed=seed
    )
    returns_by_eps[eps_val] = rets

def moving_average(x, window=1000):
    x = np.asarray(x)
    csum = np.cumsum(np.insert(x, 0, 0.0))
    ma = (csum[window:] - csum[:-window]) / window
    return ma

WINDOW = 1000
print(f"Retorno promedio por episodio, ultimos {WINDOW} episodios (episodio {epi
for eps_val in epsilon_values:
    rets = returns_by_eps[eps_val]
    first = np.mean(rets[:WINDOW])
    last = np.mean(rets[-WINDOW:])
    std_last = np.std(rets[-WINDOW:])
    print(f"eps={eps_val:.2f}  retorno_inicial~{first:7.3f}  retorno_final~{last
```

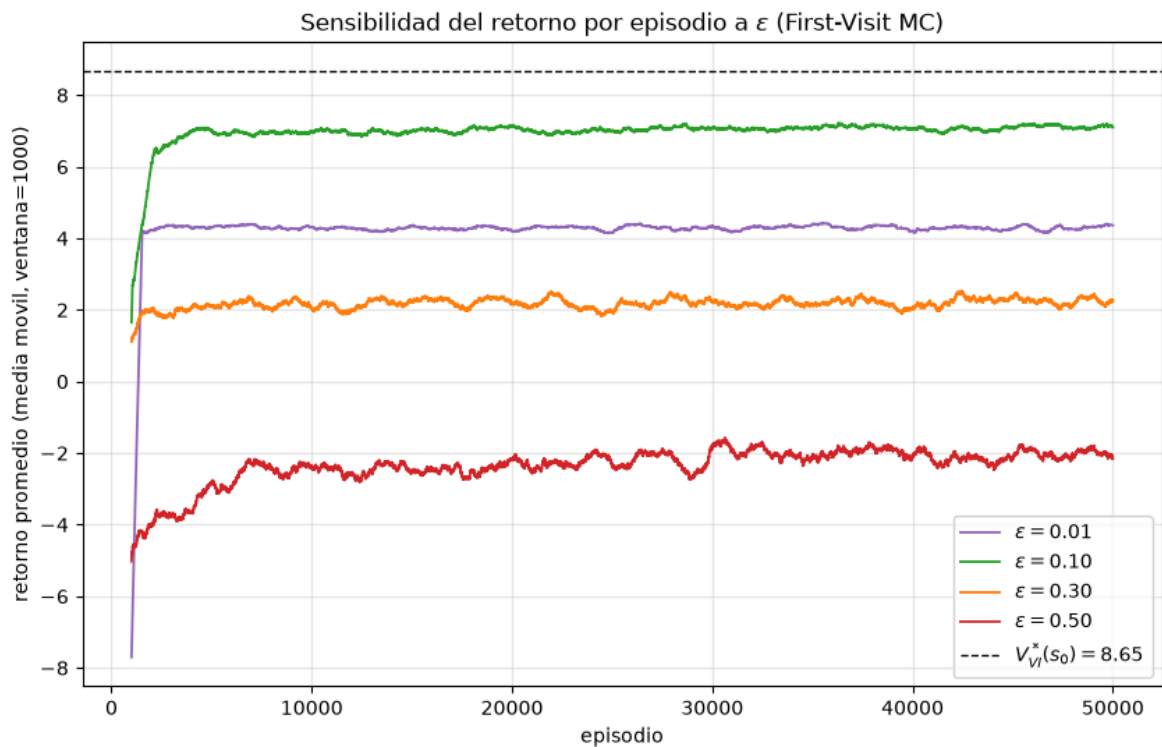
Retorno promedio por episodio, ultimos 1000 episodios (episodio 49000 a 50000):

eps=0.01	retorno_inicial~	-7.702	retorno_final~	4.361	std_final=	1.255
eps=0.10	retorno_inicial~	1.650	retorno_final~	7.101	std_final=	2.078
eps=0.30	retorno_inicial~	1.109	retorno_final~	2.253	std_final=	3.726
eps=0.50	retorno_inicial~	-5.023	retorno_final~	-2.162	std_final=	5.458

```
In [9]: fig, ax = plt.subplots(figsize=(10, 6))
colors = {0.01: "#9467bd", 0.10: "#2ca02c", 0.30: "#ff7f0e", 0.50: "#d62728"}

for eps_val in epsilon_values:
    ma = moving_average(returns_by_eps[eps_val], WINDOW)
    x = np.arange(WINDOW, episodes + 1)
    ax.plot(x, ma, label=f"$\\varepsilon$={eps_val:.2f}$", color=colors.get(eps_v

ax.axhline(V_vi[s0], color="black", linestyle="--", linewidth=1, label=f"$V^*_{s0}$")
ax.set_xlabel("episodio")
ax.set_ylabel(f"retorno promedio (media movil, ventana={WINDOW})")
ax.set_title("Sensibilidad del retorno por episodio a $\\varepsilon$ (First-Visi
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



Respuesta

Si domina uno: $\epsilon = 0.10$, con retorno promedio final ~ 7.1 contra ~ 4.3 ($\epsilon = 0.01$), ~ 2.3 ($\epsilon = 0.30$) y ~ -2.2 ($\epsilon = 0.50$). La relación no es monótona, hay un máximo interior. Con muy poco ϵ (0.01) la cobertura es insuficiente y la política se estanca por falta de datos (el mismo problema de cobertura del Inciso 1 de la Tarea 2, ahora visible directamente en el retorno). Con mucho ϵ (0.30, 0.50) el retorno reportado es el de la política ϵ -soft completa, que paga en cada episodio el costo de sus acciones aleatorias contra una trampa de -10 y un límite de 200 pasos.

¿Generaliza? El patrón cualitativo (existe un óptimo intermedio) sí, pero el valor 0.10 no. Depende de la escala de las penalizaciones (una trampa más suave toleraría ϵ mayores), del tamaño del espacio de estados (más grande exige más exploración o un cronograma decreciente para no atascarse como con 0.01), y del horizonte del episodio (más largo penaliza más cada desvío aleatorio). Por eso en la práctica se prefiere decaer ϵ durante el entrenamiento en vez de fijar un valor único.

Inciso 4

Investigación bibliográfica: busquen y lean un paper publicado entre 2020 y 2025 que aplique métodos Monte Carlo o sus extensiones a un problema real. El paper debe ser de una fuente indexada: NeurIPS, ICML, ICLR, JMLR, o similar. Escriban un resumen técnico de media página que incluya: el problema que resuelve, como aplica o extiende Monte Carlo, los resultados principales, y una reflexión sobre qué limitaciones de Monte Carlo clásico que discutimos esta semana resuelve o no resuelve ese trabajo.

Respuesta

Paper: Mankowitz, D.J., Michi, A., Zhernov, A., et al. (2023). "*Faster sorting algorithms discovered using deep reinforcement learning*". **Nature**, 618, 257-263 (Google DeepMind; misma familia de rigor editorial que NeurIPS/ICML, y el metodo predecesor AlphaZero se publico en Science).

Problema: encontrar rutinas de ordenamiento mas rapidas que las de `libc++` para arreglos pequeños (3-5 elementos) usados como caso base en algoritmos mas grandes, ejecutadas billones de veces al dia.

Extension de Monte Carlo: formulan la construccion del programa de ensamblador como un juego de un jugador y corren AlphaZero: Monte Carlo Tree Search guiado por una red de politica y valor, en vez del rollout aleatorio hasta el final del episodio que usa MC clasico. La red de valor evita esperar el retorno completo (bootstrapping entre simulaciones) y la red de politica concentra las simulaciones en ramas prometedoras en lugar de explorar uniformemente.

Resultados: algoritmos hasta 70% mas rapidos para 3-5 elementos, integrados formalmente a `libc++` de LLVM (primer cambio en la rutina en mas de una decada), y una tecnica de hashing relacionada desplegada en Abseil (Google), usada trillones de veces al dia.

Reflexion: si resuelve la varianza del rollout puro (la red de valor acorta el horizonte efectivo de cada simulacion, igual que motiva mezclar MC con TD). No resuelve el costo de muestreo: el espacio de busqueda se describe como comparable en tamaño a las posiciones de Go, y entrenar exige computo fuera del alcance de casi cualquier equipo -- la misma cota del coleccionista de cupones del Inciso 1 de la Tarea 2, a una escala astronomicamente mayor. Tampoco resuelve el problema de tareas continuas: el metodo sigue siendo episodico, solo que el dominio elegido ya termina de forma natural.

Inciso 5

Dictamen tecnico: redacten un parrafo dirigido al equipo directivo de la empresa de videojuegos argumentando si Monte Carlo es una solucion viable para generar comportamiento de personajes en el dominio descrito, cuales son sus limitaciones principales en ese contexto especifico, y que metodologia recomendarian como paso siguiente considerando lo que saben sobre Temporal-Difference Learning.

Respuesta

Monte Carlo es viable como prueba de concepto -- en segundos de computo obtuvimos un personaje que llega a la meta, recoge recompensas en el camino y conserva variabilidad entre partidas gracias a la politica ϵ -soft -- pero no lo recomendamos para produccion tal como esta. Nuestros propios resultados muestran las dos limitaciones que importan al negocio: la politica solo coincide con la optima en 72.8% de los estados, y los errores se concentran justo en las zonas menos exploradas del mapa, un problema que se agrava en niveles mas grandes que nuestra cuadrícula de 6x6; y Monte Carlo solo actualiza al terminar el episodio completo, sin forma nativa de aprender de partidas

interrumpidas o de reentrenar en producción con datos de jugadores reales.

Recomendamos Temporal-Difference Learning (SARSA o Q-Learning, eventualmente TD(λ)) como siguiente paso: al actualizar con bootstrapping paso a paso reduce la varianza que en nuestro experimento dejaba pares (s, a) con un solo dato dominando el error, aprende de trayectorias parciales, y escala mejor a niveles mas grandes o a personajes que deban ajustarse en vivo.